Studio Nx

JS 305 — 오디오 회귀 테스트 비교 지표

2026-10-21

JavaScript 305 — 비교 지표 (`tests/audio/lib/metrics.mjs`)

항목	내용
파일	`studionx/tests/audio/lib/metrics.mjs`
작업	S1-12 오디오 회귀 테스트 기반: 두 렌더링이 같은 소리인지 판정하는 지표 (PRD 00 §7.1)
작성	Claude (AI), 2026-10-21, 사용자 검토
이전, 다음	이전 304 장치 화면의 Sampler 연결, 다음 306 실행 도구

1. 하는 일

현재 렌더링과 골든을 세 가지 숫자로 비교하고, 허용 오차 안이면 통과로 판정합니다. 브라우저가 필요 없는 순수 계산이라 Node에서 바로 시험합니다.

지표	계산	기본 허용	잡아내는 것
샘플 최대 차이	모든 샘플에서 두 값의 차이 절댓값의 최댓값	0.001 (-60 dBFS)	한 군데만 크게 달라진 경우 (노트 하나, 클릭 소리)
차이 RMS	두 신호의 차이를 만들어 제곱평균제곱근을 dBFS로	-80 dBFS	전체적으로 조금씩 달라진 경우 (필터, 이펙트 계수)
LUFS 차이	통합 라우드니스(ITU-R BS.1770)의 차이	0.05 LU	사람이 느끼는 음량이 달라진 경우
최소 피크	현재 렌더링의 피크	0.001	소리가 아예 안 나는 경우

```
compareRenders(현재, 골든, 샘플레이트, 허용오차)
  길이가 다르면 —▶ 바로 실패
  피크가 너무 작음 —▶ 실패 (빈 렌더링이 빈 골든과 같다고 통과하는 일을 막음)
  diffMetrics —▶ 최대 차이, 차이 RMS
  integratedLufs —▶ 현재와 골든의 LUFS, 그 차이
  하나라도 허용 오차를 넘으면 failures 에 이유를 적고 pass = false
```

2. 언제 쓰나

쓰는 곳	하는 일
`run.mjs` (307)	프로젝트마다 현재 렌더링과 골든을 비교합니다.
`selftest.mjs` (307)	지표가 맞는지, 일부러 바꾼 소리를 잡는지 확인합니다.

3. 코드

3.1 단순 계산

```
export function toDb(x) {
  return x > 0 ? 20 * Math.log10(x) : -Infinity;
}
```

```
export function peakOf(channels) {
  let p = 0;
  for (const ch of channels) for (let i = 0; i < ch.length; i++) { const v = Math.abs(ch[i]); if (v > p)
p = v; }
  return p;
}
```

```
export function diffMetrics(a, b) {
  let maxDiff = 0, sumSq = 0, n = 0;
  for (let c = 0; c < a.length; c++) {
    const x = a[c], y = b[c];
    for (let i = 0; i < x.length; i++) {
      const d = x[i] - y[i];
      const ad = Math.abs(d);
      if (ad > maxDiff) maxDiff = ad;
      sumSq += d * d;
      n += 1;
    }
  }
  return { maxDiff: maxDiff, rmsDiffDb: toDb(Math.sqrt(sumSq / Math.max(1, n))) };
}
```

3.2 통합 라우드니스

```
function kWeightingStages(fs) {
  // 1단계: 높은 음을 올리는 셸빙 필터 (머리 주변의 음향 효과)
  const f1 = 1681.974450955533, g1 = 3.999843853973347, q1 = 0.7071752369554196;
  let k = Math.tan(Math.PI * f1 / fs);
  const vh = Math.pow(10, g1 / 20);
  const vb = Math.pow(vh, 0.4996667741545416);
  let a0 = 1 + k / q1 + k * k;
  const s1 = {
    b: [(vh + vb * k / q1 + k * k) / a0, 2 * (k * k - vh) / a0, (vh - vb * k / q1 + k * k) / a0],
    a: [2 * (k * k - 1) / a0, (1 - k / q1 + k * k) / a0],
  };
  // 2단계: 낮은 음을 깎는 하이패스 필터
  const f2 = 38.13547087602444, q2 = 0.5003270373238773;
  k = Math.tan(Math.PI * f2 / fs);
  a0 = 1 + k / q2 + k * k;
  const s2 = { b: [1 / a0, -2 / a0, 1 / a0], a: [2 * (k * k - 1) / a0, (1 - k / q2 + k * k) / a0] };
  return [s1, s2];
}
```

```
function biquad(x, st) {
  const y = new Float64Array(x.length);
  let z1 = 0, z2 = 0;
  const b0 = st.b[0], b1 = st.b[1], b2 = st.b[2], a1 = st.a[0], a2 = st.a[1];
  for (let i = 0; i < x.length; i++) {
    const v = x[i];
    const o = b0 * v + z1;
    z1 = b1 * v - a1 * o + z2;
    z2 = b2 * v - a2 * o;
    y[i] = o;
  }
```

```
  return y;
}
```

```
export function integratedLufs(channels, sampleRate) {
  const stages = kWeightingStages(sampleRate);
  const filtered = channels.map(function (ch) {
    let x = ch;
    stages.forEach(function (st) { x = biquad(x, st); });
    return x;
  });
  const len = filtered[0].length;
  const block = Math.round(0.4 * sampleRate);
  const step = Math.round(0.1 * sampleRate);
  const blocks = [];
  for (let start = 0; start + block <= len; start += step) {
    let z = 0;
    for (const ch of filtered) {
      let s = 0;
      for (let i = start; i < start + block; i++) s += ch[i] * ch[i];
      z += s / block;
    }
    blocks.push(z);
  }
  const lufsOf = function (z) { return -0.691 + 10 * Math.log10(z); };
  const abs = blocks.filter(function (z) { return z > 0 && lufsOf(z) > -70; });
  if (!abs.length) return -Infinity;
  const meanAbs = abs.reduce(function (s, z) { return s + z; }, 0) / abs.length;
  const gate = lufsOf(meanAbs) - 10;
  const rel = abs.filter(function (z) { return lufsOf(z) > gate; });
  if (!rel.length) return -Infinity;
  return lufsOf(rel.reduce(function (s, z) { return s + z; }, 0) / rel.length);
}
```

3.3 판정

```
export function compareRenders(current, golden, sampleRate, tolerance) {
  const tol = Object.assign({}, DEFAULT_TOLERANCE, tolerance || {});
  const failures = [];
  const peak = peakOf(current);
  if (current.length !== golden.length || current[0].length !== golden[0].length) {
    failures.push('길이가 다름: 현재 ' + current[0].length + ' 프레임, 골든 ' + golden[0].length + ' 프레
임');
    return { pass: false, failures: failures, maxDiff: NaN, rmsDiffDb: NaN, lufsCurrent: NaN, lufsGolde
n: NaN, lufsDiff: NaN, peak: peak };
  }
  if (peak < tol.minPeak) failures.push('소리가 거의 없음: 피크 ' + peak.toExponential(2));
  const d = diffMetrics(current, golden);
  const lc = integratedLufs(current, sampleRate);
  const lg = integratedLufs(golden, sampleRate);
  const lufsDiff = lc === lg ? 0 : Math.abs(lc - lg);
  if (d.maxDiff > tol.maxDiff) failures.push('샘플 최대 차이 ' + d.maxDiff.toExponential(2) + ' > ' + to
l.maxDiff);
  if (d.rmsDiffDb > tol.rmsDb) failures.push('차이 RMS ' + d.rmsDiffDb.toFixed(1) + ' dBFS > ' + tol.rmsD
b + ' dBFS');
  if (!(lufsDiff <= tol.lufs)) failures.push('LUFS 차이 ' + lufsDiff.toFixed(3) + ' LU > ' + tol.lufs + '
```

```
LU');
  return { pass: failures.length === 0, failures: failures, maxDiff: d.maxDiff, rmsDiffDb: d.rmsDiffDb,
lufsCurrent: lc, lufsGolden: lg, lufsDiff: lufsDiff, peak: peak };
}
```

4. 통합 라우드니스 계산

LUFS는 "사람 귀에 얼마나 크게 들리는가"를 숫자로 만든 값입니다. 단순히 샘플 값의 평균을 내면 낮은 소리와 높은 소리를 같은 비중으로 세지만, 귀는 중고음에 더 민감합니다. 그래서 먼저 K 가중 필터를 거칩니다.

단계	내용
K 가중	1단계는 높은 음을 올리는 셸빙 필터(약 +4dB, 1.7kHz 근처), 2단계는 38Hz 아래를 깎는 하이패스 필터입니다. 계수는 샘플레이트에 맞춰 식으로 계산하므로 44.1kHz와 48kHz에서 모두 맞습니다.
블록	0.4초 길이로 0.1초씩 밀면서(75% 겹침) 블록마다 두 채널의 제곱 평균을 더합니다.
절대 문턱	-70 LUFS보다 작은 블록은 버립니다. 무음 구간이 값을 끌어내리지 못하게 합니다.
상대 문턱	남은 블록의 평균에서 10 LU 낮은 값보다 작은 블록을 또 버립니다. 아주 작은 구간(페이드 아웃)이 값을 끌어내리지 못하게 합니다.
결과	남은 블록의 평균을 `-0.691 + 10 log10(평균)`으로 바꿉니다.

997Hz, 진폭 1(풀스케일) 사인을 양쪽 채널에 똑같이 넣으면 약 0 LUFS가 나옵니다. 이 값으로 구현이 맞는지 확인합니다(자체 시험).

5. 최신 문법으로 쓰면

```
// peakOf 와 diffMetrics 의 최대 차이를 최신 문법으로 쓴 모양
export const peakOf = (channels) => Math.max(0, ...channels.map((ch) => ch.reduce((m, v) => Math.max(m,
Math.abs(v)), 0)));

const maxDiff = Math.max(...a.map((x, c) => x.reduce((m, v, i) => Math.max(m, Math.abs(v - b[c][i])),
0)));

// 허용 오차 병합
const tol = { ...DEFAULT_TOLERANCE, ...tolerance };
```

전통 문법 (실제 코드)	최신 문법
`for` 문으로 최댓값 찾기	`reduce`와 `Math.max`
`Object.assign({}, A, B)`	객체 전개 `{ ...A, ...B }`
`function (z) { return ... }`	화살표 함수

여기서 `Math.max(...배열)`은 샘플이 수십만 개이면 호출 스택을 넘길 수 있어서 실제 코드는 `for` 문을 씁니다.

6. 왜 이렇게

선택	이유
지표를 셋으로 나눴습니다	한 가지만 쓰면 놓치는 변화가 있습니다. RMS만 보면 한 군데의 클릭 소리를, 최대 차이만 보면 전체에 퍼진 미세한 변화를, 둘 다 사람이 느끼는 음량 변화는 LUFS 없이 판단하기 어렵습니다. PRD §7.1이 정한 세 가지입니다.

선택	이유
LUFS 허용을 0.05 LU로 아주 좁게 잡았습니다	같은 코드는 같은 소리를 내므로(차이가 1e-7 수준) 좁게 잡아도 오탐이 없고, 믹서 값이 0.1dB만 어긋나도 잡을 수 있습니다. 실험으로 마스터 볼륨 -0.1dB 변경을 실제로 잡는 것을 확인했습니다.
소리가 없으면 별도로 실패시킵니다	렌더링이 통째로 실패해 무음이 나오고 골든도 무음이면 차이가 0이라 통과해 버립니다. 이 사고가 가장 위험하므로 최소 피크를 따로 둡니다.
길이가 다르면 지표를 계산하지 않고 바로 실패입니다	길이가 다르면 샘플을 맞춰 비교할 수 없고, 길이가 바뀐 것 자체가 큰 변화(템포 계산, 꼬리 길이)이기 때문입니다.
LUFS를 직접 구현했습니다	외부 라이브러리 없이 100줄 안팎이고, 샘플레이트별 계수 계산과 두 문턱의 동작을 자체 시험으로 확인할 수 있습니다.
허용 오차를 덮어쓸 수 있게 했습니다	이후 플러그인(WASM 비교 등)은 허용 오차가 더 넓어야 할 수 있습니다. 프로젝트 JSON의 `tolerance`로 프로젝트마다 정합니다.

7. 시험

`studionx/tests/audio/selftest.mjs`의 첫 부분이 브라우저 없이 확인합니다.

- `toDb`: 1은 0dB, 0.1은 -20dB, 0은 `-Infinity`입니다. `peakOf`는 모든 채널의 절댓값 최댓값입니다.
- LUFS: 44.1kHz와 48kHz 모두 997Hz 풀스케일 사인은 약 0 LUFS이고, 진폭 0.1은 정확히 20 LU 낮습니다.
- 문턱: 뒤쪽 무음이나 -40dB 구간의 길이가 2초든 10초든 값이 같고, 무음을 붙여도 0.5 LU 넘게 내려가지 않습니다. 완전한 무음과 400ms보다 짧은 소리는 `-Infinity`입니다.
- `diffMetrics`: 모든 샘플이 0.001 달라지면 최대 0.001, RMS -60dBFS입니다. 같은 신호는 최대 0, RMS `-Infinity`입니다.
- `compareRenders`: 같으면 통과, 0.99배(0.087dB)만 작아도 LUFS로 실패, 1e-6 잡음은 통과, 둘 다 무음이면 실패, 길이가 다르면 실패, 허용 오차를 덮어쓸 수 있습니다.
- 시험 중 발견한 일: 처음에는 "무음을 붙여도 값이 같다"고 기대했는데, 소리와 맞닿은 경계 블록(일부만 소리가 든 블록)이 값에 들어가서 약 0.3 LU 달랐습니다. 표준 알고리즘이 원래 그렇게 동작하고 구현은 맞으므로, 기대값을 "붙인 구간의 길이가 달라도 값이 같다"로 고쳤습니다.